KORA NEWSROOM · LIVRAISON V1 · FEUILLE DE ROUTE

Anne-Laure
Semaine du 9 au 13 novembre 2026

À lire avec le **plan commun** (périmètre, règles, ordre de fusion, recette).
Aya est ta référente toute la semaine : si tu bloques plus de 30 minutes sur un point, appelle-la.

1. Ta semaine en bref

Tes trois correctifs font partie de la V1, et deux d'entre eux corrigent des anomalies classées **P1** (à corriger avant toute ouverture à des utilisateurs réels). Le fond de ton travail est bon et bien testé ; ce qui reste à faire, c'est surtout de **resserrer chaque PR sur son sujet** pour qu'elle passe la CI et se relise facilement.

Quand	PR	Ce qu'il reste à faire
Lundi 10 h	Relire la PR 0 d'Aya	Grille de relecture du plan commun (section 14). 20 minutes.
Lundi 10 h 30	**P2-09** — fausse source directe	Corriger aussi `evidence.ts`, ajouter un test, mettre à jour `docs/11` et quatre autres documents, ouvrir la PR.
Lundi 14 h	**#5** — exports CSV (P1-06)	Mettre la branche à jour, retirer ce qui est hors sujet, corriger le cas de la tabulation, mettre à jour `docs/11`.
Mardi 9 h	Relecture de #5 et P2-09 avec Aya	Présenter chaque PR en 5 minutes (section 6).
Mardi, après la fusion de #5	Mise à jour de P2-09	Un conflit certain sur `docs/11`, résolution exacte en 3.2, étape 5.
Mardi 14 h	**#1** — sorties HTTP (P1-08)	Mettre à jour, résoudre un conflit de test, retirer les fichiers générés, formater, documenter les nouveaux codes d'erreur.
Mercredi 9 h	Test de collecte de #1 avec Aya	Tu conduis le test comparatif (plan commun, section 9).
Mercredi après-midi	Relire #3 d'Aya ; préparer la recette	Grille de relecture ; feuille de recette.
Jeudi	Recette	Partie locale le matin, partie production l'après-midi.

2. Avant de commencer (lundi, 15 minutes)

2.1 Pourquoi tes CI étaient rouges

Ce n'était pas ton code. Sur `main`, ESLint échoue sur des fichiers **régénérés automatiquement par la plateforme** (`src/integrations/supabase/*`, `src/integrations/lovable/appUserConnector.ts`). Pour avoir une CI verte, tu les as reformatés dans chaque branche : c'était logique, mais ces fichiers sont réécrits par la plateforme à chaque synchronisation, et ce reformatage crée du bruit et des conflits. Lundi matin, la PR 0 d'Aya règle le problème une fois pour toutes sur `main`. Ensuite, tes branches doivent revenir à la version de `main` pour tous les fichiers qui ne concernent pas leur sujet.

2.2 Les règles qui comptent pour toi cette semaine

- **Mettre une branche à jour = `git fetch origin` puis `git merge origin/main --no-edit`.** Jamais `git rebase`, jamais `git push --force`, jamais `git commit --amend` sur un commit déjà poussé. (Sans `--no-`

`edit`, Git ouvre un éditeur dans le terminal pour le message. Si ça arrive, accepte le message proposé : dans vim, tape `:wq` puis Entrée ; dans nano, Ctrl+X.)

- **Si Git annonce un conflit qui n'est pas prévu dans ce document :** `git merge --abort` (rien n'est perdu) et appelle Aya. Les conflits prévus sont tous décrits ici, avec leur résolution exacte.
- **Tu nommes toujours les fichiers que tu valides** : `git add <fichier>`. Jamais `git add -A`, `git add .` ni `git commit -a`. Avant chaque commit : `git status`. Si `.env` y apparaît, `git restore .env` et préviens Aya : ce fichier est versionné, et une clé ajoutée dedans serait publiée sur GitHub.
- **En local, toujours** `SUPABASE_URL= bun run dev`, jamais `bun run dev` seul : le `.env` contient l'adresse de la base de production.
- **Rien dans l'éditeur de la plateforme** cette semaine : tout passe par une branche et une PR.
- **Une PR ne contient que son sujet.** Pas de reformatage d'autres fichiers, pas de changement de version de dépendance, pas de fichier de `src/integrations/`.
- **Ne jamais supprimer un test qui gêne** : comprendre pourquoi il échoue, puis corriger le code ou l'attendu, et l'expliquer dans le commit. (Tu l'as très bien fait le 6 novembre en rétablissant ton test.)
- **Garde ta méthode** : test rouge, correction minimale, test vert, un commit par étape. C'est exactement ce qu'on attend.
- **Messages de commit en français**, qui disent ce qui change et pourquoi, sans fautes de frappe (relis avant de valider). Aucune mention d'outil de génération de code.

2.3 Environnement

À faire une seule fois, avant toute commande de ce document

Les commandes de ce document portent des commentaires en fin de ligne (`# doit afficher 30`…). Par défaut, le terminal zsh de macOS ne les reconnaît pas : il les passe à la commande, qui échoue (`git commit --no-edit # termine la fusion` donnerait une erreur). Lance une fois :

```
echo 'setopt interactivecomments' >> ~/.zshrc && source ~/.zshrc
```

Et quand tu copies depuis ce PDF : un bloc à la fois, puis compare le résultat à ce qui est annoncé en commentaire avant de passer au suivant.

```
source ~/.nvm/nvm.sh && nvm use 24     # Node 24
bun --version                           # 1.2.19 attendu

# Les cinq contrôles à lancer avant chaque push (branche à jour avec main)
git fetch origin
bun install --frozen-lockfile
bunx tsc --noEmit -p .              # 1. aucune erreur
bun run lint                        # 2. 0 erreur ; 6 avertissements react-refresh connus, à
ignorer
bun --no-env-file run test          # 3. tous verts
bun --no-env-file run build         # 4. build OK
# 5. Hygiène du diff : chacune des quatre commandes ne doit RIEN afficher
git diff --check origin/main
git diff origin/main --name-only | grep -E '^(\.env|drizzle/|db/|supabase/|src/integrations/)'
git diff origin/main | grep '^+' | grep -i -E 'cursor|grok|chatgpt|claude|codex|copilot|gemini'
git log origin/main..HEAD --format=%B | grep -i -E
'cursor|grok|chatgpt|claude|codex|copilot|gemini|co-authored'
```

Le contrôle 5 attrape ce que les tests ne voient pas : espace en fin de ligne, marqueur de conflit oublié dans un document, fichier interdit, mention d'outil de génération de code. S'il affiche quelque chose, corrige avant de pousser, ou demande à Aya.

3. Tâche 1 — P2-09 : fausse « source directe » (lundi matin)

3.1 Le problème, en entier

Ta branche `AL_P2-09_fausse_source_directe` corrige le test dans le collecteur (flux « propre » de l'entreprise). Mais l'anomalie P2-09 de `docs/11` signale le **même défaut** à un deuxième endroit : l'indicateur documentaire, dans `src/server/domain/evidence.ts` ligne 61. C'est lui qui affiche « source directe » sur un signal :

```
const isDirect = (o: Observation) =>
  !!company && (o.outlet.domain === company.domain || hostOf(o.url).endsWith(company.domain));
```

Avec ce code, un article de `notpasqal.com` compte comme source officielle de Pasqal. On corrige les deux endroits avec **une seule fonction**, pour ne pas dupliquer la règle.

3.2 Les étapes

```
git fetch origin
git switch AL_P2-09_fausse_source_directe
git pull --ff-only
```

Étape 1 — le test rouge. Crée `src/test/server/domain/evidence.test.ts` :

```
import { describe, expect, it } from "vitest";

import type { Observation } from "@/lib/api/types";
import { computeEvidence, isCompanyHost } from "../../../server/domain/evidence";

const NOW = new Date("2026-11-09T09:00:00.000Z");

function observation(url: string, outletDomain: string): Observation {
  return {
    id: `obs_${outletDomain}`,
    signalId: null,
    sourceId: null,
    companyId: "co_pasqal",
    outlet: { name: outletDomain, domain: outletDomain, brand: { initials: "X", bg: "", fg: "" }
},
    url,
    title: "Pasqal annonce un partenariat",
    excerpt: "",
    collectedAt: NOW.toISOString(),
    publishedAt: NOW.toISOString(),
    eventDate: null,
    matchConfidence: 90,
    isSyndicated: false,
  };
}

describe("source directe (P2-09)", () => {
  it("reconnaît le domaine et ses sous-domaines, pas un domaine qui se termine pareil", () => {
    expect(isCompanyHost("pasqal.com", "pasqal.com")).toBe(true);
    expect(isCompanyHost("blog.pasqal.com", "pasqal.com")).toBe(true);
    expect(isCompanyHost("pasqal.com", "www.pasqal.com")).toBe(true);
    expect(isCompanyHost("notpasqal.com", "pasqal.com")).toBe(false);
    expect(isCompanyHost("pasqal.com", "")).toBe(false);
  });

  it("ne compte pas notpasqal.com comme source officielle de pasqal.com", () => {
    const company = { name: "Pasqal", domain: "pasqal.com" };
    const impostor = observation("https://notpasqal.com/actualite", "notpasqal.com");
```

```
    expect(computeEvidence([impostor], company, NOW).directSource).toBe(false);

    const official = observation("https://www.pasqal.com/fr/actualites/x", "pasqal.com");
    expect(computeEvidence([official], company, NOW).directSource).toBe(true);
  });
});
```

```
bunx vitest run src/test/server/domain/evidence.test.ts   # doit échouer : isCompanyHost n'existe
pas encore
git add src/test/server/domain/evidence.test.ts
git commit -m "Test : un domaine qui se termine comme celui de l'entreprise n'est pas une source
directe (rouge)"
```

Étape 2 — la correction. Dans `src/server/domain/evidence.ts`, juste au-dessus de `export function hostOf`, ajoute :

```
/** Vrai si `host` est le domaine de l'entreprise ou l'un de ses sous-domaines (« notpasqal.com »
n'est pas « pasqal.com »). */
export function isCompanyHost(host: string, domain: string): boolean {
  const d = domain.toLowerCase().replace(/^www\./, "");
  return d !== "" && (host === d || host.endsWith(`.${d}`));
}
```

Puis, ligne 61, remplace `hostOf(o.url).endsWith(company.domain)` par `isCompanyHost(hostOf(o.url), company.domain)`.

Dans `src/server/adapters/collector.ts`, ajoute l'import en tête (à côté des autres imports de `../domain/`) :

```
import { isCompanyHost } from "../domain/evidence";
```

et remplace tes trois lignes

```
const host = hostOf(url);
const domain = company.domain.replace(/^www\./, "");
const own = host === domain || host.endsWith(`.${domain}`);
```

par une seule :

```
const own = isCompanyHost(hostOf(url), company.domain);
```

```
# Formatage (retire aussi la double ligne vide de ton test de collecte)
bunx eslint --fix src/server/domain/evidence.ts src/server/adapters/collector.ts \
  src/test/server/domain/evidence.test.ts src/test/server/collecte-reelle.test.ts
bunx vitest run src/test/server/domain/evidence.test.ts src/test/server/collecte-reelle.test.ts
git status                              # attendu : les 4 fichiers ci-dessous, et rien d'autre
git add src/server/domain/evidence.ts src/server/adapters/collector.ts \
  src/test/server/domain/evidence.test.ts src/test/server/collecte-reelle.test.ts
git commit -m "Source directe : même règle de domaine pour le collecteur et l'indicateur
documentaire (P2-09)"
```

> **Vérifié le 8 novembre**
> Ces modifications, appliquées à ta branche : TypeScript OK, lint 0 erreur, tous les tests verts (dont les 2 nouveaux et ton test `notkalray.eu`).

Étape 3 — `docs/11-anomalies-connues.md`. L'anomalie n'est corrigée qu'en partie : le filtre par nom des flux généralistes reste approximatif. La règle du document : une entrée partiellement corrigée ne décrit plus que ce qui reste vrai.

- Dans le tableau « Signaux, score et indicateur documentaire », remplace la ligne P2-09 par :

```
| P2-09 | **Filtre par nom des flux généralistes approximatif.** Quand un flux n'appartient pas à l'entreprise, `RealCollector` garde ses entrées par une simple recherche de sous-chaîne du nom ou des alias : sensible aux accents et sans frontière de mot, un nom court produit de faux rattachements ; la règle diffère de `matchCompany`. | [`collector.ts`](../src/server/adapters/collector.ts) (`RealCollector`) | Réutiliser la comparaison de `matchCompany` pour ce filtre. | 9/11 |
```

- Dans « Corrigées depuis la consolidation », ajoute :

```
| P2-09 (partiel) | Le test `hostOf(url).endsWith(company.domain)` oubliait le point : « notpasqal.com » comptait comme source officielle de « pasqal.com », dans l'indicateur documentaire et pour décider si un flux appartient à l'entreprise. | Une seule règle, `isCompanyHost` : le domaine exact ou un sous-domaine (`.domaine`). Corrigé le 9 novembre 2026. | [`evidence.ts`](../src/server/domain/evidence.ts), [`collector.ts`](../src/server/adapters/collector.ts) |
```

Ajoute-la **juste sous la ligne de séparation** (`|---|---|---|---|`) du tableau « Corrigées depuis la consolidation », en première ligne.

Quatre autres documents décrivent l'ancien test et deviendraient faux. Remplacements exacts (cherche le texte de gauche avec Cmd+F) :

Fichier	Remplacer	Par
`docs/05-regles-metier.md`, puce « Source directe »	ou que l'hôte de l'URL se termine par ce domaine.	ou que l'hôte de l'URL est ce domaine ou l'un de ses sous-domaines (`isCompanyHost` : `pasqal.com` ou `blog.pasqal.com`, jamais `notpasqal.com`).
`docs/06-securite-et-flux-tiers.md`, paragraphe qui commence par « Le collecteur décide aussi »	Tout le paragraphe (une seule ligne).	Le collecteur décide aussi qu'un flux est celui de l'entreprise si son hôte est le domaine de l'entreprise ou l'un de ses sous-domaines (`isCompanyHost`) : `notpasqal.com` n'est pas le site de `pasqal.com`, et ses entrées passent par le filtre par nom. L'indicateur documentaire reconnaît une source directe par la même règle (voir [Indicateur documentaire](05-regles-metier.md#indicateur-documentaire)).
`docs/03-reference-api.md`, point 4 de la collecte	(test `hostOf(url).endsWith(company.domain)`, sans point de séparation)	(domaine exact ou sous-domaine, `isCompanyHost`)
`docs/04-dictionnaire-des-donnees.md`, point 28	Tout le point 28 (une seule ligne, qui commence par « 28. **Test de domaine sans point** »).	28. **Test de domaine sans point** : corrigé le 9 novembre 2026. Le collecteur et l'indicateur documentaire utilisent `isCompanyHost` ([`domain/evidence.ts`](../src/server/domain/evidence.ts)), qui exige le domaine exact ou `.`, comme `matchCompany`.

```
# Contrôle : une seule ligne doit sortir, ta ligne « P2-09 (partiel) » de docs/11
grep -rn "endsWith(company.domain)\|point de séparation" docs src
git status                              # 5 fichiers de docs modifiés, rien d'autre
git add docs/03-reference-api.md docs/04-dictionnaire-des-donnees.md docs/05-regles-metier.md \
  docs/06-securite-et-flux-tiers.md docs/11-anomalies-connues.md
git commit -m "Documentation : P2-09 partiellement corrigée, règle isCompanyHost"
```

> **Vérifié le 8 novembre**
> Ces cinq modifications de documentation, rejouées sur une copie : aucun conflit avec #1, et la commande de contrôle ne renvoie que la ligne attendue.

Étape 4 — lundi, après la fusion de la PR 0 (vers 12 h) : mise à jour et PR.

```
git fetch origin
git merge origin/main --no-edit         # pas de conflit attendu
# les cinq contrôles (section 2.3) : tout doit passer, lint compris
git diff origin/main --stat             # attendu : 9 fichiers (collector.ts, evidence.ts, 2
tests, docs 03, 04, 05, 06, 11)
git push
```

Puis ouvre la PR sur GitHub (base `main`) avec le modèle de description du plan commun (section 7). Titre proposé : *« P2-09 : un domaine qui se termine comme celui de l'entreprise n'est plus une source directe »*. Demande la relecture d'Aya.

Étape 5 — mardi, dès qu'Aya a fusionné #5 : remettre P2-09 à jour. Ce conflit est **certain** : ta PR #5 et P2-09 ajoutent chacune une ligne juste sous l'en-tête du tableau « Corrigées » de `docs/11`.

```
git switch AL_P2-09_fausse_source_directe
git fetch origin
git merge origin/main --no-edit
# attendu : « CONFLICT (content): Merge conflict in docs/11-anomalies-connues.md », et aucun autre
```

Ouvre `docs/11-anomalies-connues.md` et cherche `<<<<<<<`. Tu dois voir exactement ceci (lignes raccourcies) :

```
<<<<<<< HEAD
| P2-09 (partiel) | Le test `hostOf(url).endsWith(company.domain)` oubliait le point …
=======
| P1-06 | La neutralisation des formules ne s'appliquait qu'aux valeurs de type chaîne …
>>>>>>> origin/main
```

Supprime **seulement les trois lignes de marqueurs** (`<<<<<<< HEAD`, `=======`, `>>>>>>> origin/main`) et garde les deux lignes de tableau, l'une sous l'autre.

```
grep -n -E '^(<<<<<<<|=======$|>>>>>>>)' docs/11-anomalies-connues.md   # ne doit RIEN afficher
grep -c '^| P2-09 (partiel)\|^| P1-06 |' docs/11-anomalies-connues.md    # doit afficher 2
git add docs/11-anomalies-connues.md
git commit --no-edit
# les cinq contrôles, puis :
git push
```

> **Vérifié le 8 novembre**
> Rejoué sur une copie : c'est le seul conflit, il a exactement cette forme, et après résolution 629 tests passent.

4. Tâche 2 — PR #5 : exports CSV, P1-06 (lundi après-midi)

À faire **après** la fusion de la PR 0. Ton correctif dans `csv.ts` est bon. Il reste trois choses : remettre la branche à jour, retirer tout ce qui est hors sujet, et corriger une petite régression.

4.1 Mise à jour et nettoyage

```
git fetch origin
git ls-tree origin/main .github/workflows/ci.yml   # doit afficher une ligne : la PR 0 est
fusionnée. Vide : STOP, attendre Aya
git switch AL_P1-06_correction_neutralisation_CSV
git pull --ff-only
git status                              # « nothing to commit, working tree clean » attendu avant
de commencer

# Temps 1 : la fusion
git merge origin/main --no-edit
# 4 conflits attendus, et aucun autre :
#   .github/workflows/ci.yml                          (ajouté des deux côtés)
#   docs/11-anomalies-connues.md
#   src/integrations/supabase/previewAuthStorage.ts
#   src/server/adapters/openai.ts
# Les 4 reprennent la version de main (ta ligne P1-06 de docs/11 sera réécrite en 4.3)
git checkout origin/main -- .github/workflows/ci.yml docs/11-anomalies-connues.md \
  src/integrations/supabase/previewAuthStorage.ts src/server/adapters/openai.ts
git status                              # plus aucune ligne « both modified » ni « both added »
git commit --no-edit                    # termine la fusion

# Temps 2 : le reste du hors-sujet reprend la version de main (et l'erreur « let timer »
disparaît)
git checkout origin/main -- src/integrations src/server/connectionKeyCrypto.ts \
  src/test/server/budget.test.ts package.json bun.lock
git status                              # seulement des fichiers de cette liste, en vert
git commit -m "Fichiers hors sujet remis à l'identique de main"

bun install --frozen-lockfile
# les cinq contrôles (section 2.3)
git diff origin/main --stat             # attendu : csv.ts et csv.test.ts, rien d'autre
```

Pourquoi deux temps : `git commit --no-edit` ne sert qu'à terminer une fusion en cours. Séparer la fusion du nettoyage évite un commit raté si Git n'a pas laissé de fusion en cours. Si le temps 2 répond « nothing to commit », c'est que tout était déjà propre : passe à la suite.

> **Pourquoi on remet `package.json` et `bun.lock` comme sur `main`**
> Ta branche figeait les versions de `drizzle-kit` et `drizzle-orm`. C'est peut-être une bonne idée, mais elle n'a rien à voir avec les exports CSV : elle mérite sa propre PR, avec une explication. Pareil pour la CI : elle arrive avec la PR 0.

4.2 La régression de la tabulation

Avant ta PR, une cellule qui commençait par une tabulation était toujours préfixée (le caractère `\t` figurait dans l'expression). Dans ta version, `trimStart()` retire la tabulation **avant** le test : `"\tvaleur"` n'est plus préfixée. Les recommandations sur l'injection de formules demandent de neutraliser aussi une cellule qui commence par une tabulation ou un retour chariot.

Test rouge : dans `src/test/server/domain/csv.test.ts`, dans le premier `describe`, ajoute :

```
it("neutralise une cellule qui commence par une tabulation, même sans formule", () => {
  const result = toCsv(["nom"], [{ nom: "\tvaleur" }]);

  expect(result).toBe("nom\r\n'\tvaleur\r\n");
});
```

```
bunx vitest run src/test/server/domain/csv.test.ts      # 1 échec attendu
git add src/test/server/domain/csv.test.ts
git commit -m "Test : une cellule qui commence par une tabulation est neutralisée (rouge)"
```

Correction : dans `sanitizeString` de `src/server/domain/csv.ts`, remplace

```
const trimmed = value.trimStart();
if (/^[=+\-@\t\r]/.test(trimmed)) {
```

par

```
if (/^[\t\r]/.test(value) || /^[=+\-@]/.test(value.trimStart())) {
```

```
bunx vitest run src/test/server/domain/csv.test.ts      # 20 tests verts
git add src/server/domain/csv.ts
git commit -m "CSV : une tabulation ou un retour chariot en tête de cellule reste neutralisé"
```

> **Vérifié le 8 novembre**
> Sur ta branche, ce test échoue sans la correction (19 verts, 1 rouge) et passe avec (20 verts).

4.3 Documentation

Dans `docs/11-anomalies-connues.md` (version de `main`, que tu viens de reprendre) :

- supprime la ligne P1-06 du tableau P1 ;
- ajoute dans « Corrigées depuis la consolidation » :

```
| P1-06 | La neutralisation des formules ne s'appliquait qu'aux valeurs de type chaîne : un
tableau (alias, thématiques) dont le premier élément était `=HYPERLINK(…)` passait tel quel. |
Neutralisation des chaînes et des tableaux, y compris derrière des espaces ; une cellule qui
commence par une tabulation ou un retour chariot est toujours préfixée ; les nombres ne sont pas
modifiés. Corrigé le 10 novembre 2026. | [`csv.ts`](../src/server/domain/csv.ts), [`csv.test.ts`]
(../src/test/server/domain/csv.test.ts) |
```

Comme pour P2-09, la ligne P1-06 se place juste sous la ligne de séparation `|---|---|---|---|` du tableau « Corrigées ».

```
git add docs/11-anomalies-connues.md
git commit -m "Documentation : P1-06 corrigée"
# les cinq contrôles
git diff origin/main --stat             # csv.ts, csv.test.ts, docs/11 : 3 fichiers
git push
```

Mets à jour la description de la PR : ajoute la correction de la tabulation, et retire la mention « ajout de la CI ». Demande la relecture d'Aya.

5. Tâche 3 — PR #1 : sorties HTTP du collecteur, P1-08 (mardi après-midi)

À faire **après** la fusion de #5 et de P2-09 (mardi midi). C'est ta plus grosse PR et elle est solide : 635 tests verts, aucune logique à reprendre. Ce qu'il reste, c'est du rangement et un peu de documentation.

5.1 Mise à jour

```
git fetch origin
git switch fix_C01_Securiser_les_sorties_HTTP_du_collecteur
git pull --ff-only
git status                              # « working tree clean » attendu
git merge origin/main --no-edit
# attendu : « CONFLICT (content): Merge conflict in src/test/server/collecte-reelle.test.ts », et
aucun autre
```

> **Ce conflit est piégeux : ne le résous pas à la main**
> Git le découpe en deux morceaux imbriqués (vers les lignes 561 et 622), dont l'un englobe plus de 300 lignes de tes tests C01. Garder « un côté » ou « les deux » à la main fait facilement perdre ou dupliquer des tests sans que tout casse. La méthode ci-dessous a été rejouée et vérifiée : on part de ta version complète du fichier, et on y recolle le seul test apporté par P2-09.

```
# 1. Reprendre ta version complète du fichier (tous tes tests C01)
git checkout --ours src/test/server/collecte-reelle.test.ts

# 2. Extraire le test de P2-09 depuis main, dans un fichier temporaire hors du dépôt
#    (commande sur 5 lignes : copie-la en entier, d'un seul bloc)
git show origin/main:src/test/server/collecte-reelle.test.ts | awk '
  /un domaine qui se termine par le domaine officiel/ { p = 1; match($0, /^ */); n = RLENGTH }
  p { print }
  p && /^ *}\);$/ { match($0, /^ */); if (RLENGTH == n) exit }
' > /tmp/test-p209.txt
wc -l < /tmp/test-p209.txt              # doit afficher 30
head -1 /tmp/test-p209.txt              # it("ne considère pas un domaine qui se termine par le
domaine officiel…
```

3. Recoller ce test au bon endroit. Ouvre `src/test/server/collecte-reelle.test.ts` et cherche `describe("C01 · intégration collecte (exercice 4b)"`. Deux lignes plus haut se trouve une ligne `});` seule, au début de la ligne : c'est la fin du bloc `describe("collecte réelle : flux des entreprises"`. Place-toi sur une ligne vide **juste au-dessus de ce `});`** et colle tout le contenu de `/tmp/test-p209.txt` (il commence par `it("ne considère pas un domaine…` et finit par `});` indenté de deux espaces). **Pas à la fin du fichier** : le test utilise `ownFeed` et `stubPage`, qui n'existent que dans ce bloc.

```
# 4. Vérifications, toutes obligatoires
grep -c notkalray src/test/server/collecte-reelle.test.ts                    # doit afficher 2
grep -n -E '^(<<<<<<<|=======$|>>>>>>>)' src/test/server/collecte-reelle.test.ts  # ne doit RIEN
afficher
bunx vitest run src/test/server/collecte-reelle.test.ts                       # 24 tests verts

# 5. Terminer la fusion
git add src/test/server/collecte-reelle.test.ts
git commit --no-edit
```

Si un des trois résultats diffère (pas 2, des marqueurs, pas 24 tests) : `git merge --abort` et appelle Aya.

```
# Ensuite, nettoyage : fichiers hors sujet, version de main (ta branche les avait reformatés)
git checkout origin/main -- src/integrations src/server/connectionKeyCrypto.ts \
  src/test/server/budget.test.ts .github/workflows/ci.yml
git status                              # seulement des fichiers de cette liste
git commit -m "Fichiers hors sujet remis à l'identique de main"

# Les 50 erreurs de formatage : correction automatique, limitée à tes fichiers
bunx eslint --fix src/server/domain/safeHttp.ts src/server/domain/security.ts \
  src/server/adapters/collector.ts src/test/server/safeHttp.test.ts \
```

```
  src/test/server/security.test.ts src/test/server/collecte-reelle.test.ts
git status                              # seulement des fichiers parmi ces six
git add src/server/domain/safeHttp.ts src/server/domain/security.ts \
  src/server/adapters/collector.ts src/test/server/safeHttp.test.ts \
  src/test/server/security.test.ts src/test/server/collecte-reelle.test.ts
git commit -m "Formatage des fichiers du correctif C01"
# les cinq contrôles
```

> **Vérifié le 8 novembre**
> Toute cette section rejouée sur une copie, après P2-09 et #5 : un seul conflit, résolu par cette méthode ; puis lint 0 erreur et 658 tests verts sur l'ensemble du dépôt.

5.2 Ce qu'on ne fait pas cette semaine

Le renommage du paramètre `timeout_ms` en `timeoutMs` dans `RealCollector` est reporté en V1.1 : il est juste, mais sans effet visible, et on ne touche pas au code qui marche la semaine de la livraison. Note-le dans la description de la PR, rubrique « Limites ».

5.3 Documentation à compléter

1. Nouveaux codes d'échec dans `docs/09-exploitation.md`, procédure « Une source est en échec ». Juste après la ligne qui commence par « Symptôme : l'alerte rouge » et se termine par « `NETWORK` (erreur réseau ou DNS, certificat invalide). », laisse une ligne vide et ajoute ce paragraphe :

```
Depuis le correctif des sorties HTTP (P1-08), le collecteur produit aussi des refus définitifs,
jamais
réessayés : `URL_REFUSED` (adresse interne, réservée ou port autre que 80/443, y compris après une
redirection), `HOST_MISMATCH` (le site redirige vers un autre domaine, par exemple `societe.com`
vers
`societe.fr` ; seul le passage avec ou sans `www.` est accepté), `TOO_MANY_REDIRECTS` (plus de 5),
`REDIRECT_NO_LOCATION` et `BODY_TOO_LARGE` (réponse de plus de 2 Mio). Pour `HOST_MISMATCH`,
ouvrir
l'adresse dans un navigateur et remplacer l'URL de la source par l'adresse finale affichée.
```

2. Ce que P2-09 a réglé. Une fois P2-09 fusionnée, cinq passages de ta documentation ne sont plus vrais : le test de domaine « avec point de séparation » est fait. Remplacements exacts (cherche le texte de gauche avec Cmd+F) :

Fichier	Remplacer	Par
`docs/06`, fin de la section SSRF	(contrôle de l'adresse obtenue si l'hébergeur le permet) et le test de domaine d'entreprise avec point de séparation (voir	(contrôle de l'adresse obtenue si l'hébergeur le permet) (voir
`docs/06`, point 12 de la liste de suivi (vers la ligne 472)	Restent la résolution DNS et le test de domaine entreprise avec point de séparation.	Reste la résolution DNS.
`docs/11`, ligne P1-08, « Restent ouverts »	; `hostOf(url).endsWith(company.domain)` sans point (P2-09)	(rien : supprimer ce morceau, point-virgule de tête compris)
`docs/11`, ligne P1-08, dernière colonne utile	DNS + adresse résolue si l'hébergeur le permet ; séparation de domaine entreprise.	DNS + adresse résolue si l'hébergeur le permet.
`docs/11`, ligne « P1-08 (partiel) » des corrigées	Rebinding DNS et séparation de domaine restent dans P1-08.	Le rebinding DNS reste dans P1-08.

```
# Contrôle : une seule ligne doit sortir, la ligne « P2-09 (partiel) » de docs/11
grep -rn "endsWith(company.domain)\|point de séparation\|séparation de domaine" docs src
git status                              # docs/06, docs/09, docs/11 modifiés, rien d'autre
git add docs/06-securite-et-flux-tiers.md docs/09-exploitation.md docs/11-anomalies-connues.md
git commit -m "Documentation : codes d'échec du collecteur, P2-09 retirée des restes de P1-08"
# les cinq contrôles
git diff origin/main --stat     # safeHttp, security, collector, leurs tests, docs : rien de
src/integrations
git push
```

> **Vérifié le 8 novembre**
> Ces cinq remplacements, rejoués sur la copie après la fusion de P2-09 : la commande de contrôle ne renvoie que la ligne attendue.

5.4 Description de la PR à mettre à jour

Ajoute à la description existante (elle est déjà très bien) :

- Le **changement de comportement des réessais** : avant, tout échec sauf 403 et 404 était réessayé ; désormais seuls `TIMEOUT`, `NETWORK`, 408, 425, 429 et 5xx le sont. Un 400, 401 ou 410 ne l'est plus, ni aucun refus de sécurité.
- La **politique `HOST_MISMATCH`**, et le fait qu'aucune des 30 sources « Site officiel » du pilote ne redirige vers un autre domaine (vérifié le 8 novembre) : on garde ce comportement strict.
- Le tableau du test de collecte comparatif de mercredi matin.

5.5 Mercredi 9 h : le test de collecte, c'est toi qui conduis

Suis le protocole du plan commun, section 9 : le script `test-collecte.sh`, d'abord sur `main`, puis sur ta branche, en local, sans clé payante. Avant chaque passage, vérifie le mode (`{"mode":"demo"}`). Ce qu'on attend :

☐ Mêmes statuts et mêmes nombres sur les deux passages pour Kalray, Keolabs, Digigram, Teem Photonics, HydroQuest et Naver Labs Europe.
☐ Keolabs (qui redirige `/feed/` vers `/feed`) : toujours un succès.
☐ Arturia : `NO_FEED` sur les deux.
☐ Alpao : échec `NETWORK` ou `TIMEOUT`, et la durée totale du script reste sous la minute.
☐ Aucun `URL_REFUSED` ni `HOST_MISMATCH` nouveau.
☐ En bas du tableau : `http://127.0.0.1/feed` REFUSÉE sur les deux ; `https://example.com:8443/feed` ACCEPTÉE sur `main` et REFUSÉE sur ta branche. C'est la preuve visible de ton correctif.

Colle les deux tableaux (fichiers de `~/kora-tests/`) dans la PR.

6. Préparer tes relectures avec Aya

En relecture, tu présentes chaque PR en 5 minutes : le problème, ta correction, le test qui échouait avant. Puis Aya te pose des questions. Ce n'est pas un examen : c'est le moment où l'on vérifie ensemble que la PR est comprise de bout en bout, pour que tu puisses la défendre devant n'importe qui et la maintenir. Voici les points à savoir expliquer avec tes mots.

PR P2-09

- Pourquoi `host.endsWith(domain)` est faux (prends l'exemple de `notpasqal.com`).
- Quel effet ce bug avait pour un journaliste (un site tiers présenté comme « source directe »).
- Pourquoi une seule fonction pour deux endroits.
- Ce qui reste de P2-09 après ta PR (le filtre par nom).

PR #5

- Pourquoi une apostrophe neutralise une formule, et pourquoi ce n'est pas une garantie absolue.
- Pourquoi le bug passait avec un tableau.
- Pourquoi `-42` (nombre) n'est pas préfixé alors que `"-42"` (chaîne) l'est.
- Ce que tu avais cassé avec `trimStart()` et comment le test le montre.
- Pourquoi `toBe(…)` est un meilleur attendu que `toContain("'")`.

PR #1

- Ce qu'est une attaque SSRF, et pourquoi `redirect: "manual"` est indispensable (exemple : une adresse publique qui redirige vers `169.254.169.254`).
- Pourquoi un seul délai pour toute la chaîne de redirections (ton exercice 4a).
- Pourquoi lire le corps en flux plutôt qu'avec `res.text()`.
- Pourquoi un refus de sécurité n'est jamais réessayé, et un délai dépassé oui (exercice 4b).
- Ce qu'est le DNS rebinding, pourquoi ta PR ne le règle pas, et ce qui limite le risque aujourd'hui (Cloudflare).
- Pourquoi tu as supprimé puis rétabli un test les 5 et 6 novembre.
- Quel cas légitime `HOST_MISMATCH` bloque, et ce que doit faire l'administratrice.

7. Relire les PR d'Aya (formation à la relecture)

Lundi 10 h, la PR 0 ; mercredi après-midi, #3 (logos). Utilise la grille du plan commun (section 14). Pour chacune, laisse au moins un commentaire : une question, une remarque ou une approbation argumentée. Pistes :

- **PR 0** : le réglage ESLint désactive-t-il seulement le formatage, ou d'autres règles aussi ? Quels fichiers exactement ? La CI se lance-t-elle sur toutes les branches ?
- **#3** : que se passe-t-il sans identifiant Brandfetch, avec un domaine invalide, si le service répond 404 ou ne répond pas ? Quelle information part chez Brandfetch quand un utilisateur ouvre l'écran Entreprises ? La documentation `docs/06` le dit-elle ?

8. Jeudi : la recette

- **9 h–11 h, partie locale** (plan commun, section 10, lignes L0 à L6), sur `main` à jour, lancée avec `SUPABASE_URL= bun run dev --port 5199`. Commence toujours par L0. Note chaque résultat : OK, ou KO avec une capture.
- **14 h–17 h, partie production** : lignes P2, P4, P7, P8, P9, avec ton propre compte (invitation envoyée par Amine). Ne lance pas de collecte, ne crée, ne modifie et ne supprime aucune donnée, et ne publie rien sur LinkedIn : en production, on regarde seulement.
- 17 h : bilan avec Aya et Amine.

9. Aide-mémoire Git

Situation	Commande
Voir où j'en suis	`git status` puis `git log --oneline -10`
Mettre ma branche à jour avec `main`	`git fetch origin` puis `git merge origin/main --no-edit`
Valider des modifications	`git status`, puis `git add <fichier> <fichier>`, puis `git commit -m "…"`. Jamais `add -A`, `add .` ni `commit -a`.
`.env` apparaît dans `git status`	`git restore .env` (ou `git restore --staged .env` puis `git restore .env` s'il est déjà ajouté), et prévenir Aya.

Récupérer ce qui a été poussé sur ma branche	`git pull --ff-only` (refuse de fusionner en silence si l'historique a divergé : dans ce cas, appelle Aya)
Voir ce que ma PR change par rapport à `main`	`git diff origin/main --stat`, puis `git diff origin/main -- <fichier>`
Remettre un fichier comme sur `main`	`git checkout origin/main -- <fichier ou dossier>`
Pendant une fusion, prendre la version de `main` pour un fichier en conflit	`git checkout origin/main -- <fichier>` (le fichier est alors aussi marqué résolu)
Pendant une fusion, prendre ma version pour un fichier en conflit	`git checkout --ours <fichier>` puis `git add <fichier>`
Pendant une fusion, garder les deux versions	Éditer le fichier, garder les deux blocs, retirer les marqueurs, puis `git add <fichier>`
Terminer une fusion	`git commit --no-edit`
Abandonner une fusion qui tourne mal	`git merge --abort` (rien n'est perdu)
Annuler une modification locale pas encore validée	`git restore <fichier>`
J'ai validé un commit, pas encore poussé, avec une erreur	Corriger, puis un nouveau commit. (`--amend` n'est permis que sur un commit jamais poussé ; dans le doute, nouveau commit.)
Annuler un commit déjà poussé	`git revert <sha>` puis `git push`. Jamais `reset` + `push --force`.
Lancer un seul fichier de tests	`bunx vitest run <chemin du fichier>`
Corriger le formatage d'un fichier	`bunx eslint --fix <fichier>`

10. Ta définition de « terminé » pour chaque PR

- ☐ Branche à jour avec `main` par fusion.
- ☐ `git diff origin/main --stat` ne montre que le sujet de la PR.
- ☐ Les cinq contrôles passent en local, y compris le contrôle 5 (hygiène du diff) qui n'affiche rien ; la CI GitHub est verte.
- ☐ Un test échouait avant ta correction et passe après, avec un commit pour chaque étape.
- ☐ `docs/11` est à jour (et `docs/06`, `docs/09` ou `docs/10` si le comportement visible change).
- ☐ La description de la PR suit le modèle : problème, changement, tests, limites, comment vérifier.
- ☐ Tu sais expliquer chaque ligne du diff à Aya.